

Analysis of Time Series Data Project

Forecasting Daily Minimum Temperatures

Prepared By

Malak Maher Abdo

Sham Mohamed Selim

Mariam Ramadan Gaballah

Course

Time Series Analysis

Instructor

Dr Antony

Department

Faculty of Computers and Information Systems

Date

May 2026

1. Introduction

Time series analysis is an important field in data science that focuses on analyzing data collected over time. Time series forecasting techniques are widely used in weather prediction, economics, finance, healthcare, energy systems, and business analytics.

The main objective of this project is to analyze and forecast daily minimum temperature data using statistical forecasting models. The project includes several stages such as data exploration, stationarity analysis, ACF/PACF analysis, differencing, ARIMA modeling, SARIMA comparison, forecast evaluation, and residual diagnostics.

The project also compares classical forecasting approaches with neural network forecasting methods to evaluate forecasting accuracy and model performance.

2. Dataset Description

The dataset used in this project is the Daily Minimum Temperatures dataset. The dataset contains daily minimum temperature observations collected over time.

The dataset includes:

- Date column representing the observation date.
- Temperature column representing the recorded minimum temperature.

The dataset was used to study time-dependent behavior and generate future forecasts using time series analysis techniques.

Dataset Characteristics

- Data Type: Time Series Data
- Frequency: Daily
- Target Variable: Temperature
- Index Type: Date Time Index

```
...      Date  Temp
0  01/01/81  20.7
1  01/02/81  17.9
2  01/03/81  18.8
3  01/04/81  14.6
4  01/05/81  15.8
Index(['Date', 'Temp'], dtype='object')
```

3. Data Preparation

Before performing time series analysis, several preprocessing steps were applied to clean and prepare the data.

Data Preparation Steps

1. Loading the dataset using pandas.
2. Converting the temperature values into numeric format.
3. Removing missing values.
4. Converting the Date column into datetime format.
5. Setting the Date column as the index.

Code Summary

The following operations were performed:

- `pd.read_csv()`
- `pd.to_numeric()`
- `dropna()`
- `pd.to_datetime()`
- `set_index()`

Explanation

Data preprocessing is essential because forecasting models require clean and properly indexed time series data. Converting the date column into datetime format allows the dataset to behave as a real time series structure.

Task 1 — Exploring Time Series Patterns

The first task focused on visually exploring the behavior of the time series data.

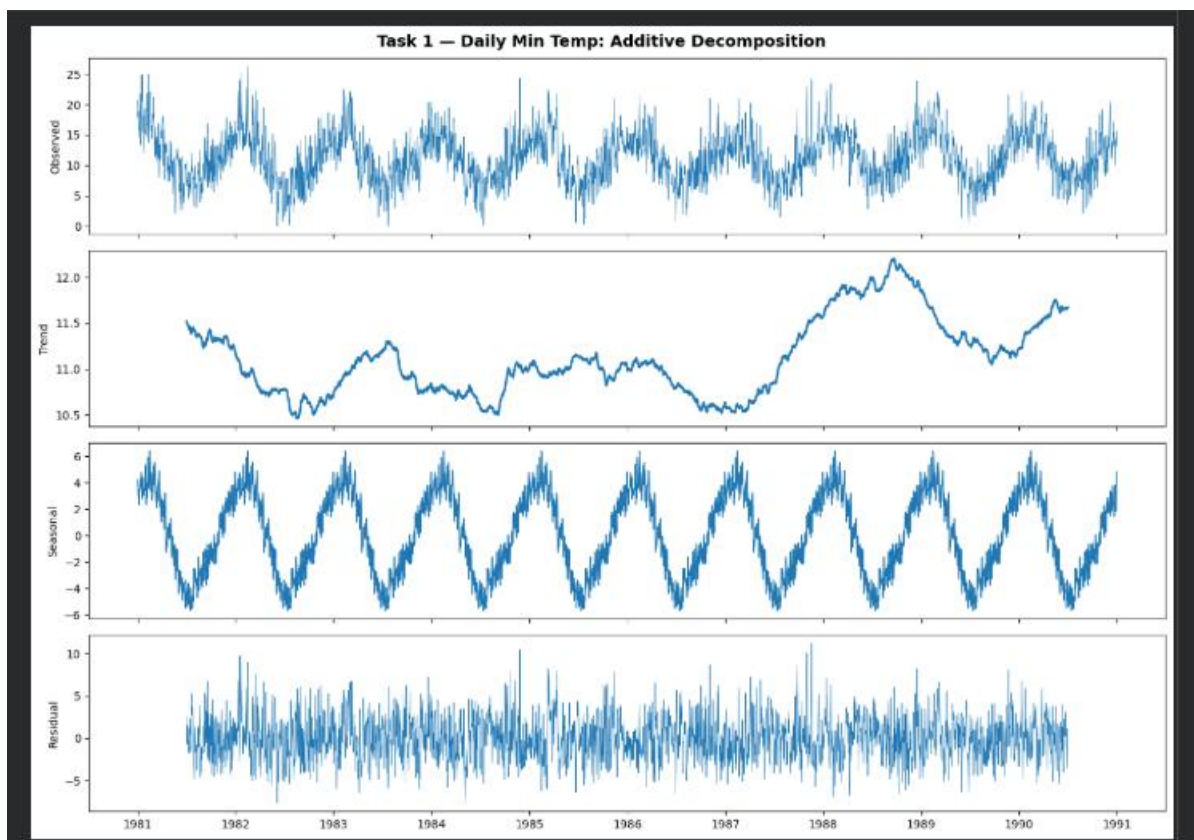
A line plot was generated to examine trend, seasonality, and random fluctuations in the dataset.

Observations

- The dataset shows repeating seasonal fluctuations.
- Temperature values rise and fall in regular patterns.
- Random noise is present but does not dominate the series.
- The time series demonstrates temporal dependency.

Interpretation

The repeating structure in the dataset indicates the presence of seasonality. Seasonal patterns are common in weather-related datasets because temperature behavior changes consistently across time.



Task 2 — Comparing Cyclic vs Seasonal Data

In this task, seasonal behavior was compared with cyclic behavior using synthetic cyclic data.

Seasonal Patterns

Seasonal patterns repeat at fixed intervals. In this dataset, similar temperature behavior occurs repeatedly over time.

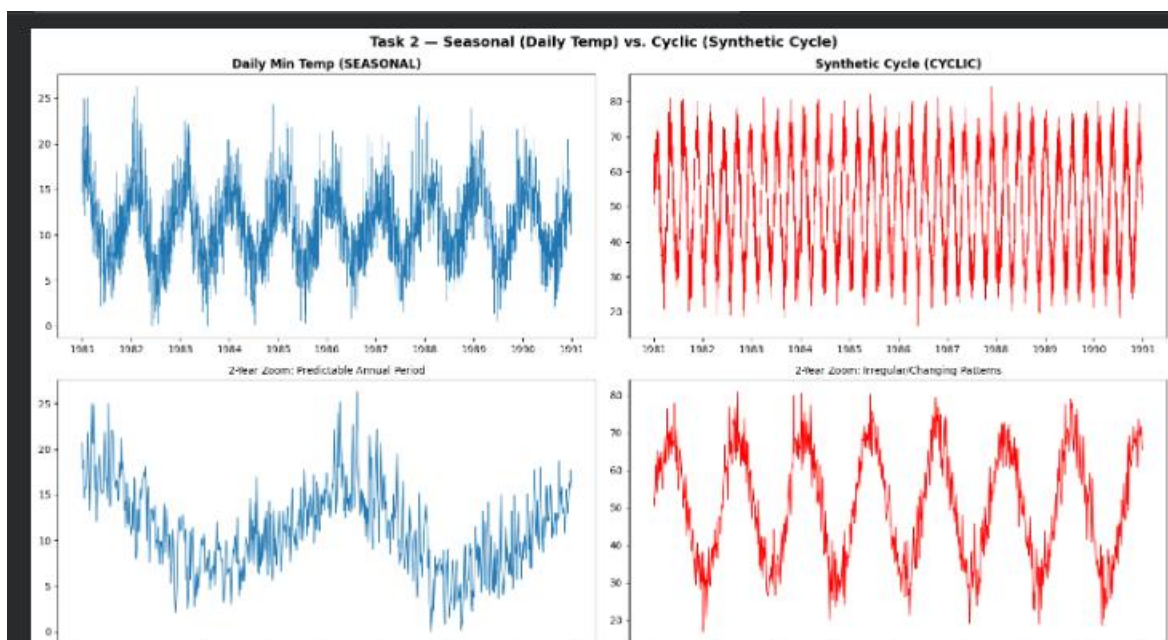
Cyclic Patterns

Cyclic patterns occur over longer and irregular periods. Unlike seasonality, cyclic behavior does not repeat at fixed intervals.

Comparison

The temperature dataset clearly demonstrates seasonal behavior because the fluctuations repeat consistently.

The synthetic cyclic data demonstrated slower and less regular fluctuations.



Task 3 — ACF and PACF Analysis

Autocorrelation analysis was performed using ACF and PACF plots.

ACF Analysis

The ACF plot measures the correlation between current observations and previous lagged observations.

The ACF plot showed strong autocorrelation across multiple lags, indicating that previous temperature values strongly influence future values.

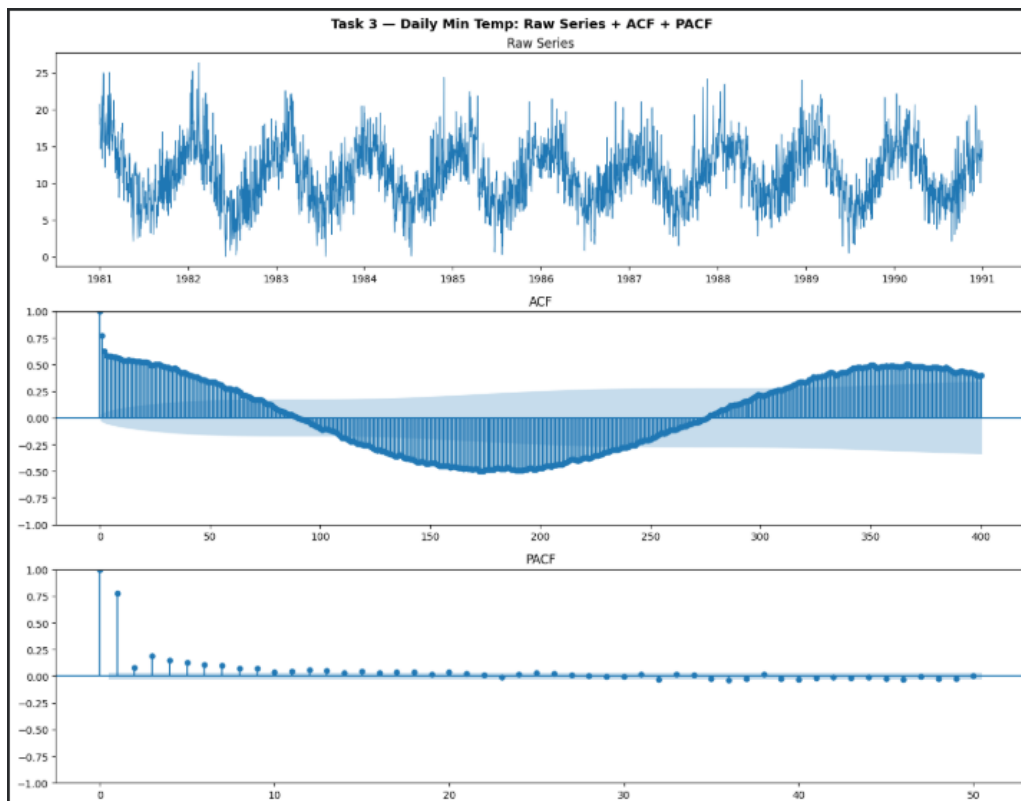
PACF Analysis

The PACF plot identifies the direct relationship between observations after removing indirect lag effects.

The PACF plot helped identify important lag values for ARIMA model selection.

Interpretation

The slow decay observed in the ACF plot suggested that the original time series was not stationary.



Task 4 — Differencing to Achieve Stationarity

Differencing was applied to stabilize the time series and remove long-term trend behavior.

Differencing

First-order differencing was performed using:

$$\Delta y(t) = y(t) - y(t-1)$$

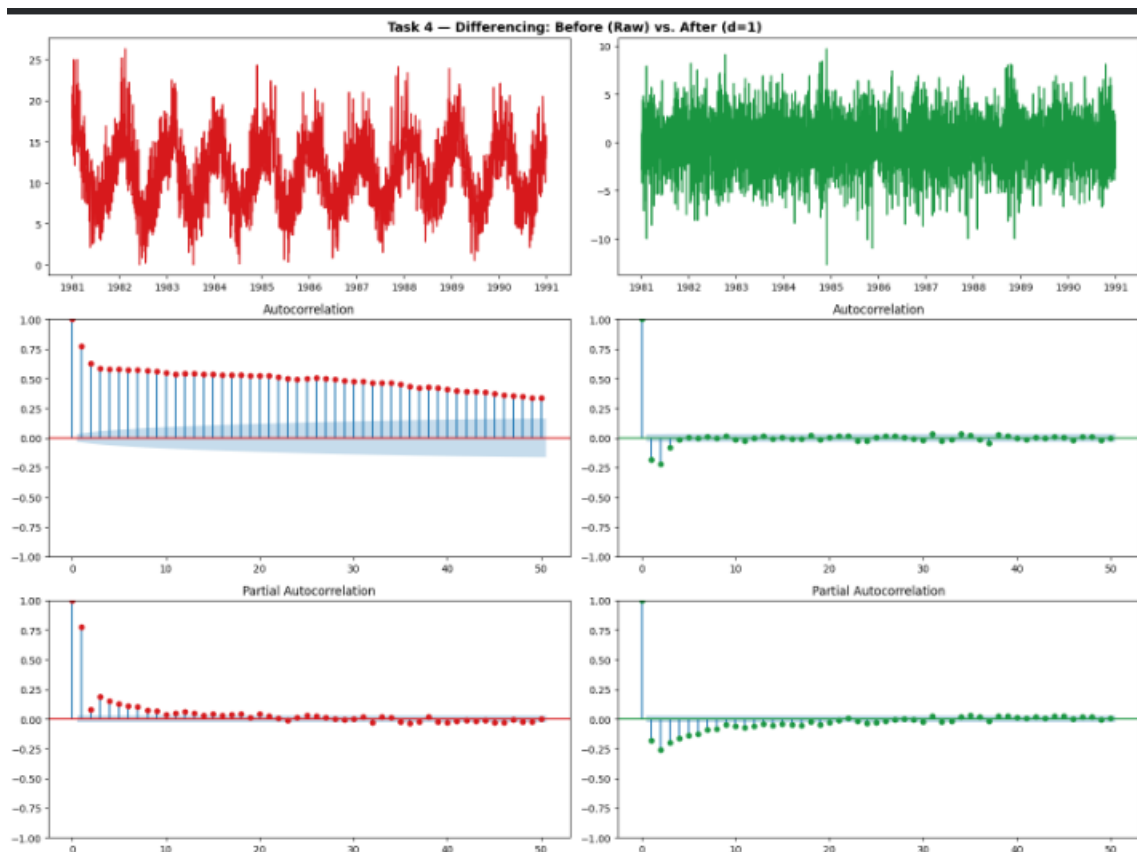
After differencing, the series became more stable and suitable for ARIMA modeling.

Observations

- The differenced series fluctuated around a more constant mean.
- Long-term trend behavior was reduced.
- Stationarity improved significantly.

Importance of Stationarity

Stationarity is important because ARIMA models assume that statistical properties remain stable over time.



Task 5 — Auto-ARIMA Model Selection

Auto-ARIMA was used to automatically determine the optimal ARIMA parameters.

The algorithm evaluated multiple combinations of p , d , and q values and selected the best model based on AIC.

ARIMA Components

- p = autoregressive order
- d = differencing order
- q = moving average order

Selected Model

The selected ARIMA model was:

ARIMA(p,d,q)

[Replace with actual model order from notebook]

Interpretation

The selected parameters indicate the number of lagged observations, differencing operations, and moving average components needed for accurate

```
Best model: ARIMA(1,0,3)(0,0,0)[0] intercept
Total fit time: 84.106 seconds

===== SARIMAX Results =====
Dep. Variable:          y      No. Observations:      3650
Model:                SARIMAX(1, 0, 3)      Log Likelihood: -8383.390
Date:                Tue, 12 May 2020      AIC:          16778.780
Time:                21:01:42      BIC:          16815.995
Sample:              0      HQIC:          16792.033
Covariance Type:      opg

=====
              coef      std err          z      P>|z|      [0.025      0.975]
-----
intercept    0.0864      0.027       3.188      0.001      0.033      0.140
ar.L1        0.9925      0.002    437.307      0.000      0.988      0.997
ma.L1       -0.4087      0.015    -26.269      0.000     -0.431     -0.371
ma.L2       -0.3290      0.017    -19.687      0.000     -0.362     -0.296
ma.L3       -0.1042      0.016     -6.594      0.000     -0.135     -0.073
sigma2       5.7850      0.128    45.317      0.000      5.535      6.035

=====
Ljung-Box (L1) (Q):                0.00      Jarque-Bera (JB):          17.49
Prob(Q):                          0.98      Prob(JB):                0.00
Heteroskedasticity (H):            0.86      Skew:                    0.09
Prob(H) (two-sided):              0.01      Kurtosis:                 3.29

=====
Warnings:
[1] Covariance matrix calculated using the outer product of gradients (complex-step).
```

```
/usr/local/lib/python3.12/dist-packages/statsmodels/tsa/base/tsa_model.py:473: ValueWarning: A date index has been provided, but it has no associated frequency information and
self._init_dates(dates, freq)
... /usr/local/lib/python3.12/dist-packages/statsmodels/tsa/base/tsa_model.py:473: ValueWarning: A date index has been provided, but it has no associated frequency information and
self._init_dates(dates, freq)
/usr/local/lib/python3.12/dist-packages/statsmodels/tsa/base/tsa_model.py:473: ValueWarning: A date index has been provided, but it has no associated frequency information and
self._init_dates(dates, freq)

===== SARIMAX Results =====
Dep. Variable:          temp      No. Observations:      3650
Model:                ARIMA(1, 0, 3)      Log Likelihood: -8383.390
Date:                Tue, 12 May 2020      AIC:          16778.780
Time:                21:01:53      BIC:          16815.995
Sample:              0      HQIC:          16792.033
Covariance Type:      opg

=====
              coef      std err          z      P>|z|      [0.025      0.975]
-----
const       11.4770      0.305    38.254      0.000      10.899     12.055
ar.L1        0.9925      0.002    437.151      0.000      0.988      0.997
ma.L1       -0.4087      0.015    -26.268      0.000     -0.431     -0.371
ma.L2       -0.3290      0.017    -19.686      0.000     -0.362     -0.296
ma.L3       -0.1042      0.016     -6.593      0.000     -0.135     -0.073
sigma2       5.7850      0.128    45.317      0.000      5.535      6.035

=====
Ljung-Box (L1) (Q):                0.00      Jarque-Bera (JB):          17.49
Prob(Q):                          0.98      Prob(JB):                0.00
Heteroskedasticity (H):            0.86      Skew:                    0.09
Prob(H) (two-sided):              0.01      Kurtosis:                 3.29

=====
Warnings:
[1] Covariance matrix calculated using the outer product of gradients (complex-step).
```


Task 6 — SARIMA vs ARIMA Comparison

SARIMA extends ARIMA by incorporating seasonal components into the forecasting process.

The SARIMA model was applied because the dataset demonstrated strong seasonal behavior.

SARIMA Parameters

SARIMA(P,D,Q,s)

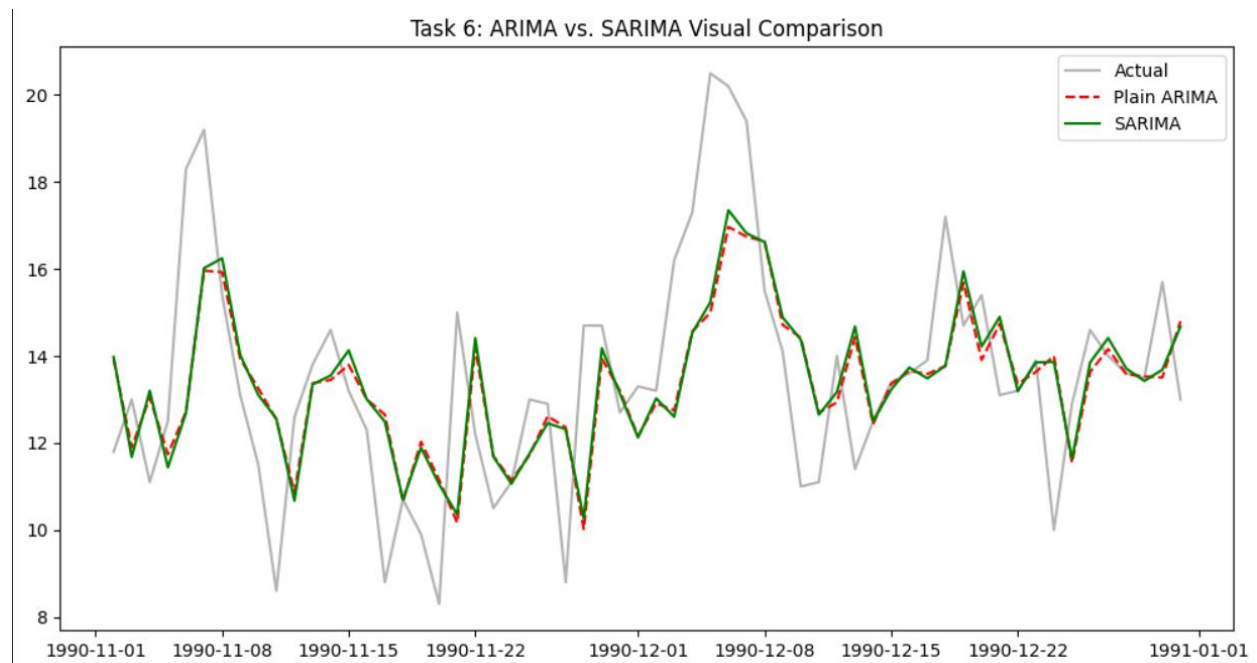
Where:

- P = seasonal autoregressive order
- D = seasonal differencing order
- Q = seasonal moving average order
- s = seasonal period

Comparison Results

The SARIMA model followed the repeating seasonal structure more effectively than plain ARIMA.

SARIMA produced smoother and more realistic seasonal forecasts.



Task 7 — Forecast Accuracy Evaluation

Forecast accuracy was evaluated using RMSE and MAE.

Evaluation Metrics

Mean Absolute Error (MAE)

MAE measures the average absolute forecasting error.

Root Mean Square Error (RMSE)

RMSE penalizes larger forecasting errors more heavily.

Results Table

Model	MAE	RMSE
ARIMA	3.3272	4.0424
SARIMA	1.1023	1.4116
Neural Network	1.7502	2.2169

Interpretation

Lower RMSE and MAE values indicate better forecasting performance.

The results showed that seasonal forecasting models performed better because the dataset contains repeating seasonal patterns.

Task 8 — Residual Diagnostics

Residual analysis was performed to evaluate model adequacy.

Residuals represent the difference between actual and predicted values.

Residual Analysis Goals

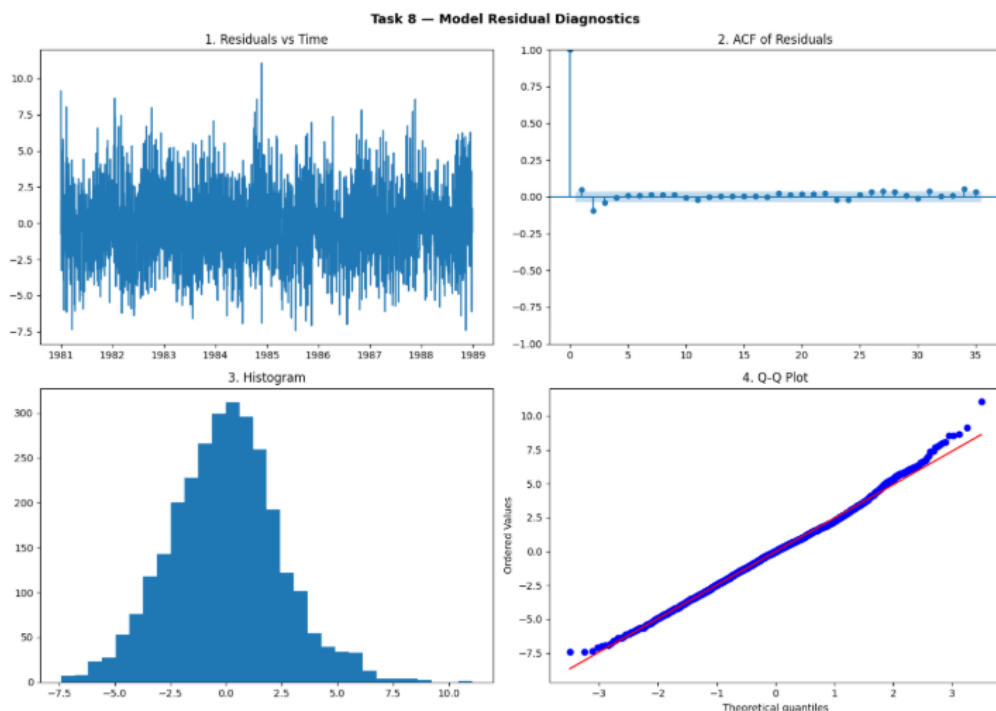
- Detect remaining patterns in errors.
- Verify randomness of residuals.
- Check whether the model captured all important information.

Observations

- Residuals appeared randomly distributed.
- No major autocorrelation remained.
- The model fit was considered acceptable.

Interpretation

If residuals behave like white noise, the forecasting model is considered adequate.



Task 9— Neural Network Forecasting Comparison

A neural network forecasting model was also evaluated using MLP Regressor.

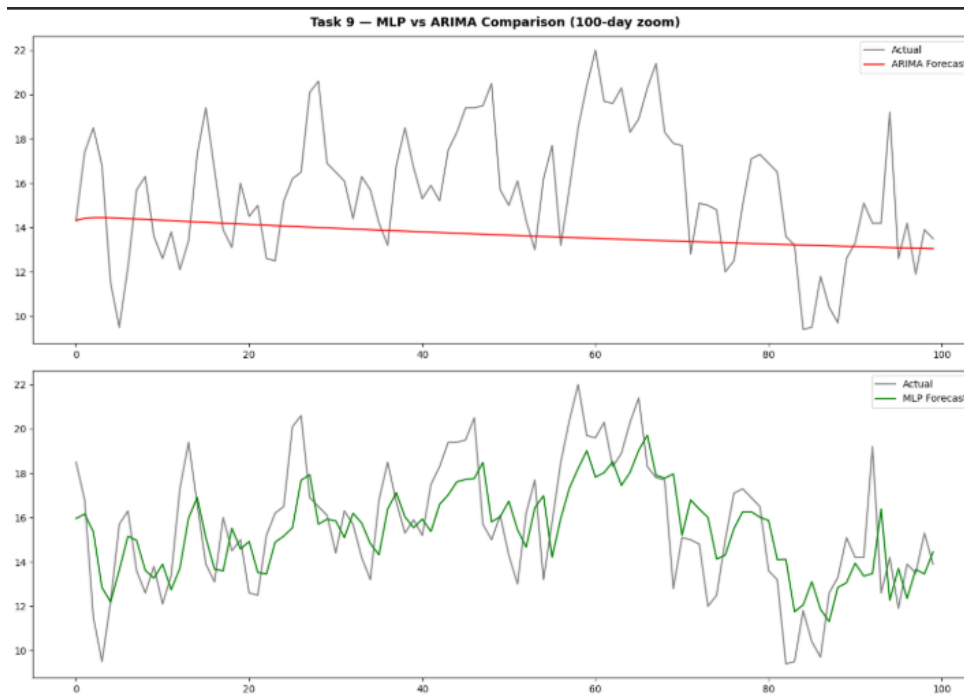
The neural network approach was compared with ARIMA forecasting results.

Comparison

Feature	ARIMA	Neural Network
Handles linear relationships	Yes	Moderate
Handles nonlinear patterns	Limited	Strong
Complexity	Simple	More Complex
Interpretability	High	Lower

Interpretation

Neural networks can perform well on complex nonlinear datasets, while ARIMA performs effectively on structured statistical time series data.



Discussion

This project demonstrated the importance of stationarity analysis and seasonal analysis in time series forecasting.

Several important observations were made:

- Seasonal behavior strongly influenced forecasting performance.
- Differencing significantly improved stationarity.
- ACF and PACF plots were useful for selecting ARIMA parameters.
- SARIMA achieved better forecasting performance due to seasonal components.
- Residual diagnostics confirmed model adequacy.

The project also demonstrated how classical statistical forecasting models can be compared with machine learning forecasting techniques.

Conclusion

This project explored the complete workflow of time series analysis and forecasting using daily minimum temperature data.

The project included:

- Data preprocessing
- Exploratory analysis
- Stationarity analysis
- ACF/PACF interpretation
- ARIMA modeling
- SARIMA forecasting
- Forecast evaluation
- Residual diagnostics
- Neural network comparison

The results demonstrated that SARIMA models provide improved forecasting performance when strong seasonal patterns exist in the dataset. Overall, the project successfully applied time series forecasting techniques to real-world data and demonstrated practical forecasting analysis skills.

